UniClaw — Open-Source Agent Platform With a 27-Adapter Provider Ontology

Apache-2.0 / MIT · github.com/uniclaw-labs/uniclaw · Architect and maintainer

27 vendors with incompatible semantics, normalized to one contract — a new brand is a plugin, not a fork.

WHAT IT IS

An open-source natural-language vehicle-control platform. A user says what they want in plain language, on whichever channel they already use, and an agent translates that intent into the right call against whichever vendor happens to own the car. A Tauri v2 desktop app (React 19 / TypeScript) on a FastAPI (Python) backend.

THE HARD PART — ONTOLOGY, NOT INTEGRATION

Twenty-seven vehicle brands expose fundamentally incompatible surfaces: different auth models, different verbs, different state machines, different notions of what "locked" or "climate on" even means. The naive build is 27 special cases and a growing pile of conditionals.

Instead I defined the **canonical capability contract** first — what a vehicle *is* to the system, independent of vendor — then made every brand a **provider adapter behind a pluggable registry** mapping its dialect onto that contract. Adding a brand is registering an adapter; the agent reasons against the canonical model and never learns a vendor's quirks. The same pattern extends to the interaction surface: **12 messaging channels** plus Alexa and Google Home, normalized to one intake contract, so a new channel is also a registration.

MULTI-MODEL ROUTING BY COST TIER

The agent runs in three modes, chosen by what the request actually deserves: **Claude Sonnet** for open-ended natural-language intent · **local Gemma via Ollama** for private, offline, or cost-sensitive operation · a **zero-cost command menu** for deterministic requests that never needed a model at all. The third mode matters most architecturally — knowing when *not* to invoke a model is part of designing a system that produces reliable output at a defensible cost.

CORRECTNESS AND REPRODUCIBILITY AS FIRST-CLASS CONCERNS

- **4,400+ automated tests** across a polyglot codebase (Python, TypeScript, Rust via Tauri) — the guarantee that generated and refactored code stays semantically correct as the surface grows.
- **OAuth2 with scoped JWTs** and per-request authorization; **Fernet-encrypted credentials** at rest; non-root containers; Docker Compose for reproducible local runs.
- Permissive licensing and a contribution surface designed to take outside adapters.

WHY IT MAPS TO THIS MISSION

The ontology-and-registry discipline the mission asks for, done in public: define the canonical model before writing integrations, make expansion a registration rather than a rewrite, route across model tiers on purpose, and back it with a test surface large enough that AI-assisted change stays safe. It's also evidence of how I'd work in the open-source library — I maintain this repo the way I'd contribute agents to soofi.xyz.

*Also published open source: a **Lever MCP server** — a reusable, governed action surface exposed as MCP.*